Tree-Based Methods

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Recap

- Queremos predecir y en funcion de observables $(\mathbf{x}_i)$

$$y = f(\mathbf{x}_i) + u \tag{1}$$

- donde la estimación de f implica la que mejor generalize (prediga mejor fuera de muestra):

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{2}$$

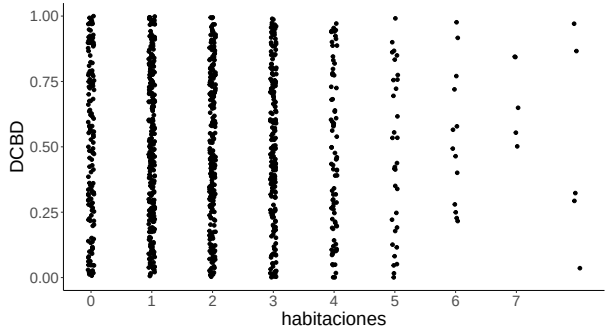
Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Motivación

- Queremos predecir:

$$Precio = f(\text{habitaciones}, \text{ubicación}) \tag{3}$$



Árboles

- Podemos asumir f es lineal

$$f(\mathbf{x}_i) = \beta_0 + \beta_1 Habitaciones_i + \beta_2 DCBD_i + u_i \tag{4}$$

Árboles

- Podemos asumir f es lineal

$$f(\mathbf{x}_i) = \beta_0 + \beta_1 Habitaciones_i + \beta_2 DCBD_i + u_i \tag{4}$$

- o ajustar un modelo flexible e interpretable como son los arboles

$$f(x_i; \{R_j, \gamma_j\}_1^J) = T(x_i; \{R_j, \gamma_j\}_1^J) \tag{5}$$

- donde

$$T(x_i; \{R_j, \gamma_j\}_1^J) = \sum_{j=1}^{J} \gamma_j I(x \in R_j) \tag{6}$$

Árboles

- El problema de optimización

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{7}$$

- es ahora

$$\{\hat{R}_j, \hat{\gamma}_j\}_1^J = \underset{\{R_j, \gamma_j\}_1^J}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, T(\mathbf{x}_i; \{R_j, \gamma_j\}_1^J)) \right\} \tag{8}$$

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

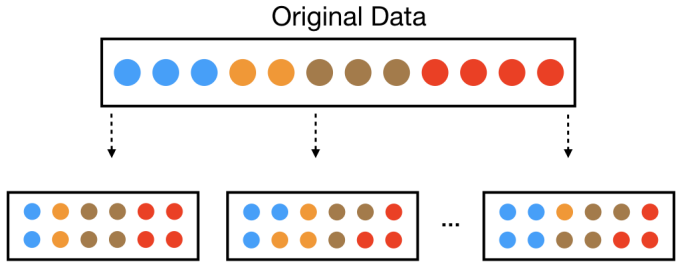
Bagging

- Problema con CART: pocos robustos.
- Idea: podemos mejorar mucho el rendimiento mediante la agregación

Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).

Bagging



Bagging

- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

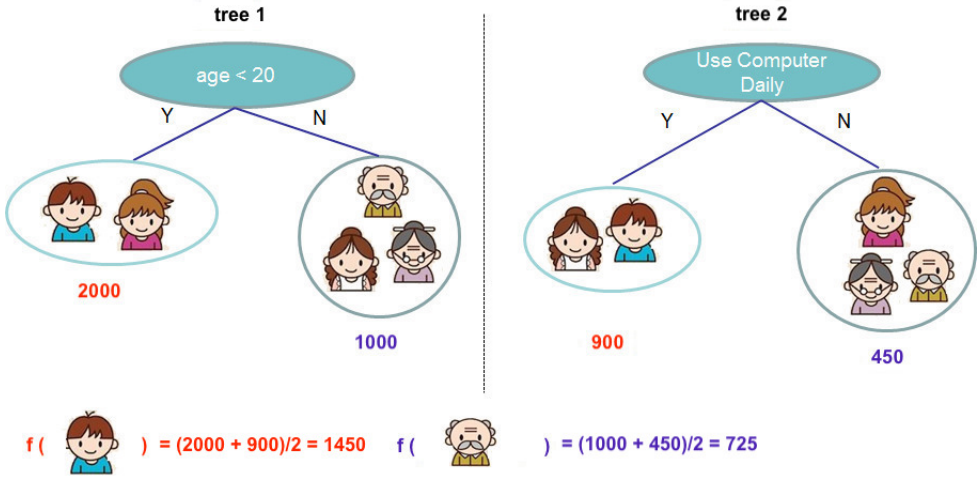
$$\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x) \tag{9}$$

Bagging

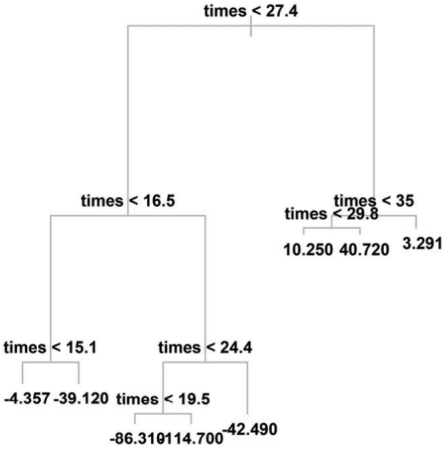
- Bagging:
  - Obtenga muestras aleatorias $(X_i^b, Y_i^b)_{i=1}^N$ de la muestra observada (bootstrap).
  - Para cada muestra, ajuste un árbol de regresión $\hat{f}^b(x)$
  - Promedie las muestras de bootstrap

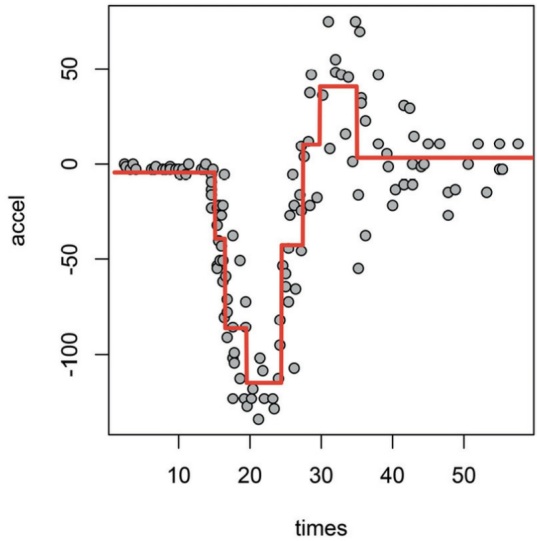
$$\hat{f}_{bag} = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x) \tag{9}$$

Bagging

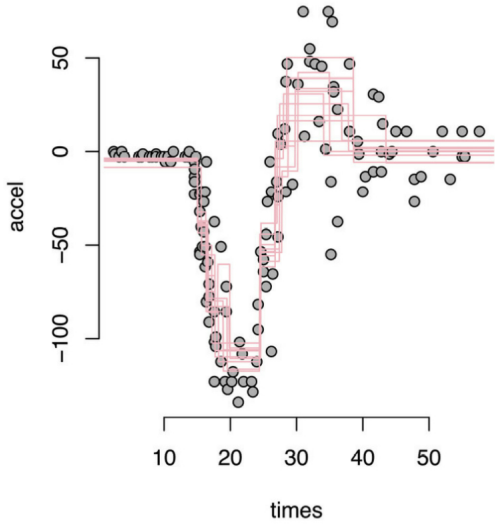

tree 1
age < 20
Y
N
2000
1000
tree 2
Use Computer Daily
Y
N
900
450
f ( ) = (2000 + 900)/2 = 1450
f ( ) = (1000 + 450)/2 = 725

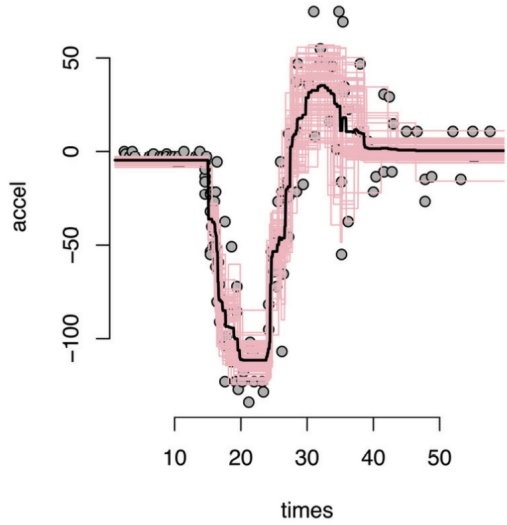
Bagging


times < 27.4
times < 16.5
times < 35
times < 29.8
10.250
40.720
3.291
times < 15.1
times < 24.4
-4.357
-39.120
times < 19.5
-86.310
-14.700
-42.490

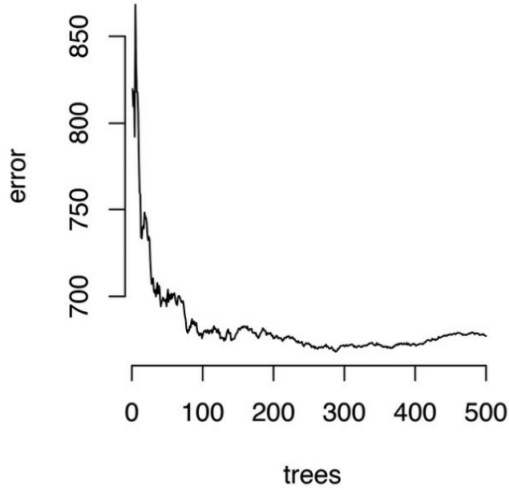

accel
50
0
-50
-100
10
20
30
40
50
times

Bagging


accel
50
0
−50
−100
10
20
30
40
50
times


accel
50
0
−50
−100
10
20
30
40
50
times

Bagging



Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Random Forests

- Problema con el bagging: si hay un predictor dominante en la muestra de entrenamiento, diferentes árboles son muy similares entre sí.
- Bosques (forests): reduce la correlación entre los árboles en el boostrap.
- Si hay k predictores, en cada partición use solo $m < k$ predictores, elegidos al azar.
- Bagging es forests con $m = k$ (usando todo los predictores en cada partición).
- m es un hiper-parámetro, $m = \sqrt{(k)}$ es un benchmark

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Bagging & RF

Out-of-Bag Error: Intuición

- En Bagging, entrenamos múltiples modelos sobre muestras bootstrap.
- Cada muestra contiene $\sim$63% de los datos originales. (Quiz)
- Estas observciones no usadas se llaman Out-of-Bag (OOB).

Bagging & RF

Out-of-Bag Error: Matemáticamente

Sean:

- $\{(x_i, y_i)\}_{i=1}^n$: conjunto de entrenamiento.
- $T_1, \ldots, T_B$: arboles entrenados en cada muestra bootstrap.
- $\mathcal{B}_i \subset \{1, \ldots, B\}$: índices de modelos donde x_i no fue usado (OOB).

Entonces, el error OOB es:

$$E_{OOB} = \frac{1}{n} \sum_{i=1}^{n} L \left(y_i, \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} T_b(x_i) \right)$$

- L función de pérdida (error cuadrático, $0-1$, etc.).
- Estima el **error de generalización, fuera de muestra** sin necesidad de un set de validación.

Bagging & RF

Importancia de Variables por Permutación: Intuición

- En modelos tipo Random Forests, queremos saber:

  ¿Qué tan importante es cada variable para las predicciones?

- La idea:
  - Medimos cuánto empeora el desempeño del modelo si rompemos la relación entre una variable y el objetivo.
- ¿Cómo rompemos esa relación?
  - Permutando los valores de la variable, dejando las otras igual.
- Si la variable es importante, el error debería aumentar.

Bagging & RF

Importancia por Permutación: Definición

- Sea E_{OOB}: error del modelo usando los datos OOB.
- Para cada variable j:
  - Permutamos la columna x_j en los datos OOB.
  - Calculamos el nuevo error: $E_{\text{perm}(j)}$.
- La importancia de la variable j es:

$$\text{VI}(j) = E_{\text{perm}(j)} - E_{\text{OOB}}$$

- Si x_j es irrelevante, el error no debería cambiar.
- Si x_j es clave, el error aumentará.

Ejemplo



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/

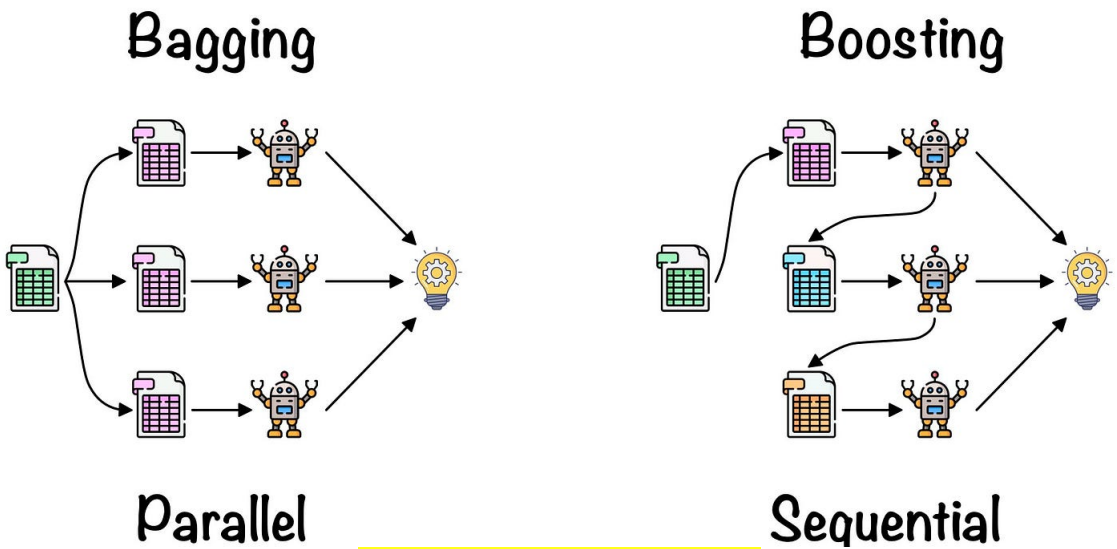
Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Boosting: Motivation

- Problema con CART: varianza alta.
- Podemos mejorar mucho el rendimiento mediante la agregación
- El boosting toma esta idea pero lo ”encara” de una manera diferente $\rightarrow$ viene de la computación
- Va a usar arboles “pequeños” y “aprende” sequencialmente

Boosting: Motivation



Boosting: Motivation

- Boosting es una de las ideas de aprendizaje más poderosas introducidas en los últimos veinte años.
- Originalmente fue diseñado para problemas de clasificación,
- Pero también puede extenderse a la regresión.

Boosting: Motivation

- Intuitivamente lo que hacen es "aprender" de los errores lentamente.
- Esto lo hace "ajustando" árboles pequeños.
- Esto permite mejorar lentamente aprendiendo $f(.)$ en áreas donde no funciona bien.
- OJO: a diferencia de *bagging*, la construcción de cada árbol depende en gran medida de los árboles anteriores

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

AdaBoost.M1

- El AdaBoost.M1 es una implementación para clasificación

AdaBoost.M1

- El AdaBoost.M1 es una implementación para clasificación
- Como viene de la computación tiene un lenguaje ligeramente diferente.
- Vocabulario:
  - $y \in \{-1, 1\}$, X vector de predictores.
  - $\hat{y} = G(X)$ (clasificador)
  - $err = \frac{1}{N} \sum_i^N I(y_i \neq G(x_i))$

AdaBoost.M1

1. Comenzamos con ponderadores $w_i = 1/N$
2. Para m = 1 hasta M:
   1. Estimar $G_m(x)$ usando ponderadores w_i .
   2. Computar el error de predicción

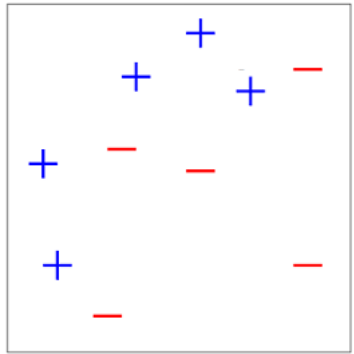
$$err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i} \tag{10}$$

   3. Obtener $\alpha_m = \ln\left[\frac{(1-err_m)}{err_m}\right]$
   4. Actualizar los ponderadores : $w_i \leftarrow w_i c_i$

$$c_i = \exp\left[\alpha_m I(y_i \neq G_m(x_i))\right] \tag{11}$$

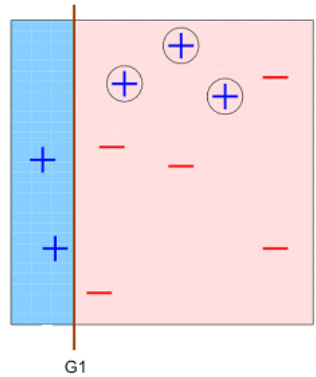
3. Resultado: $G(x) = \text{sign}[\sum_{m=1}^{M} \alpha_m G_m(x)]$

AdaBoost.M1



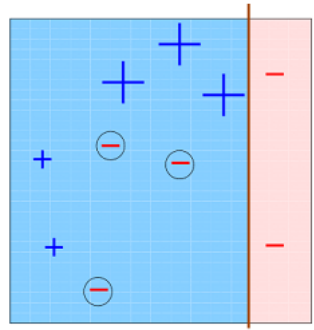
AdaBoost.M1

- Iteración 1



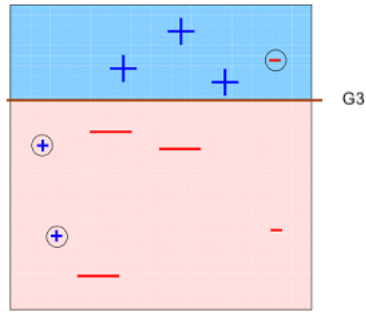
AdaBoost.M1

- Iteración 2



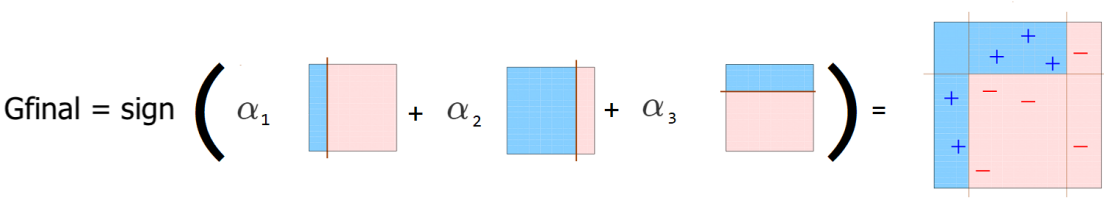
AdaBoost.M1

- Iteración 3



AdaBoost.M1

- Resultado Final



AdaBoost.M1

1. Comenzamos con ponderadores $w_i = 1/N$
2. Para m = 1 hasta M:
   1. Estimar $G_m(x)$ usando ponderadores w_i .
   2. Computar el error de predicción

$$err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i} \tag{12}$$

   3. Obtener $\alpha_m = \ln\left[\frac{(1-err_m)}{err_m}\right]$
   4. Actualizar los ponderadores : $w_i \leftarrow w_i c_i$

$$c_i = \exp\left[\alpha_m I(y_i \neq G_m(x_i))\right] \tag{13}$$

3. Resultado: $G(x) = \text{sign}[\sum_{m=1}^{M} \alpha_m G_m(x)]$

AdaBoost.M1

- $c_i = \exp\left[\alpha_m I(y_i \neq G_m(x_i))\right]$
- Si fue correctamente predicho, $c_i = 1$.
- En caso contrario, $c_i = \exp(\alpha_m) = \frac{(1-err_m)}{err_m} > 1$
- En cada paso el algoritmo da mas importancia relativa a las predicciones incorrectas.
- Paso final: promedio ponderado de estos pasos

$$G(x) = \text{sign}[\sum_{m=1}^{M} \alpha_m G_m(x)] \tag{14}$$

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

Boosting Trees

Intuition

- Boosting combina muchos modelos débiles (por ejemplo, árboles pequeños), entrenados **secuencialmente**, donde cada nuevo modelo se ajusta para **corregir los errores del modelo anterior**.
  - Los modelos débiles por sí solos tienen bajo desempeño (alto sesgo).
  - Boosting reduce el error al combinar varios de estos modelos.
  - Cada nuevo modelo se entrena sobre los errores (residuos) del conjunto anterior.
  - El resultado final es un modelo fuerte con mejor capacidad predictiva.

Boosting Trees

Algoritmo

1. Iniciamos fijando $\hat{f}(x) = 0$ y $r_i = y_i$ para todos los i del training set
2. Para $m = 1, 2, ..., M$
   1. Ajustamos un árbol $\hat{f}^m(x)$ con J hojas en (X, r)
   2. Actualizamos $\hat{f}(x)$

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^m(x) \tag{15}$$

   3. Actualizamos los residuales

$$r_i \leftarrow r_i - \lambda \hat{f}^m(x_i) \tag{16}$$

3. El modelo final es

$$\hat{f}_{boost}(x) = \sum_m \lambda \hat{f}^m(x) \tag{17}$$

Boosting Trees: Iteraciones

- Cuantas iteraciones (M) usar?
  - Cada iteración generalmente reduce el error de ajuste, de modo que para M lo suficientemente grande este error puede hacerse arbitrariamente pequeño (sesgo se va a cero).
  - Sin embargo, ajustar demasiado bien los datos de entrenamiento puede llevar a overfit (sobreajuste)
  - Por lo tanto, hay un número óptimo $M^\star$ que minimiza el error fuera de muestra
  - Una forma conveniente de encontrar $M^\star$ con validación cruzada

Boosting Trees: Hiperparámetros

- Los otros hiperparámetros a fijar son
  - λ la tasa a la que aprende, los valores típicos son 0.01 o 0.001
  - El tamaño del árbol: número de hojas J
    - $4 \leq J \leq 8$ funciona bien
    - $J = 2$ suele ser insuficiente
    - $J > 10$ rara vez se utiliza

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

De Boosting a Gradient Boosting

- Boosting: modelos secuenciales que corrigen errores del anterior.
- AdaBoost: ajusta pesos a las observaciones mal clasificadas.
- ¿Y si en lugar de ajustar pesos, minimizamos directamente una función de pérdida?

Gradient Boosting Trees

- El objetivo es

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \tag{18}$$

- Boosting: suma de M árboles, cada uno con parámetros $\Theta_m = \{R_{jm}, \gamma_{jm}\}_{j=1}^{J}$

$$f_M(x) = \sum_{m=1}^{M} T(x; \Theta_m) \tag{19}$$

- el problema es

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, \sum_{m=1}^{M} T(x_i; \Theta_m)) \right\} \tag{20}$$

Gradient Boosting Trees

- El problema

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, \sum_{m=1}^{M} T(x_i; \Theta_m)) \right\} \tag{21}$$

- Se puede aproximar por

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m)) \right\} \tag{22}$$

- Ejemplo para una función de perdida cuadrática

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} (\underbrace{y_i - f_{m-1}(x_i)}_{r_i} - T(x_i; \Theta_m))^2 \right\} \tag{23}$$

- Es decir, ajustar un árbol a los residuos $r_i = y_i - f_{m-1}(x_i)$: el algoritmo de Boosting que vimos antes

Gradient Boosting Trees

- For **arbitrary** loss functions this "simplified" problem

$$\hat{\Theta}_m = \underset{\Theta_m}{\operatorname{argmin}} \left\{ \sum_{i=1}^{n} L\left(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m)\right) \right\} \tag{24}$$

- is still a tricky problem

Gradient Boosting Trees

- Podemos resolverlo usando la idea de gradiente descendiente, observación por observación:

$$f_m(x_i) = f_{m-1}(x_i) - \lambda \underbrace{\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}}_{g_{im}} \tag{25}$$

- Esto solo actualiza f en los x_i de entrenamiento.

Gradient Boosting Trees

Algoritmo

1. Inicializar:

$$f_0(x) = \arg\min_{\gamma} \sum_{i=1}^{n} L(y_i, \gamma)$$

2. Para $m = 1$ hasta M:
   1. Calcular los pseudo-residuos:

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

   2. Ajustar un árbol de regresión a los r_{im}, obteniendo regiones terminales R_{jm}, $j = 1, \ldots, J$.
   3. Para cada región R_{jm}, calcular:

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

   4. Actualizar el modelo:

$$f_m(x) = f_{m-1}(x) + \lambda \sum_{j=1}^{J} \gamma_{jm} \cdot I(x \in R_{jm})$$

3. El modelo final es:

$$\hat{f}(x) = f_M(x)$$

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

XGBoost: Motivation

- XGBoost (*eXtreme Gradient Boosting*, Chen y Guestrin 2016) es gradient boosting con tres cambios:
  1. un objetivo **regularizado**;
  2. una aproximación de **segundo orden**
  3. código eficiente que lo hace rápido y escalable.

XGBoost: Motivation

- ¿Por qué hablar de XGBoost? Según Chen y Guestrin (2016):
  - De las 29 soluciones ganadoras publicadas en el blog de Kaggle en 2015, 17 usaron XGBoost; ocho lo usaron como único modelo. El segundo método más popular, las redes profundas, apareció en 11.
  - En la KDD Cup 2015, todos los equipos del top 10 usaron XGBoost.
  - La evidencia más reciente va en la misma dirección: para datos *tabulares* de tamaño mediano, los ensambles de árboles siguen superando a las redes profundas (Grinsztajn, Oyallon y Varoquaux 2022).
  - Para datos no estructurados (imágenes, texto), las redes son mejores (próximo tema).

XGBoost: el objetivo regularizado

- Escribimos cada árbol como $T(x) = w_{q(x)}$: q asigna x a una de las J hojas y w_j es el valor de la hoja j (los γ_j de antes)
- Objetivo **global**, sobre los M árboles:

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{m=1}^{M} \Omega(T_m), \tag{26}$$

donde

$$\Omega(T) = \zeta J + \frac{1}{2}\varphi \sum_{j=1}^{J} w_j^2 + \alpha \sum_{j=1}^{J} |w_j| \tag{27}$$

- Ω penaliza la complejidad del **árbol** (número de hojas J y tamaño de los w_j); no depende de x_i

XGBoost: la penalidad y los nombres en el paquete

Slides	Rol	`xgboost`	Conexión
ζ	costo por hoja	`gamma` (`min_split_loss`)	poda
φ	L2 sobre los w_j	`lambda`	Ridge
α	L1 sobre los w_j	`alpha`	Lasso
λ	shrinkage	`eta`	boosting

- Son las mismas penalidades de regularización: Ridge y Lasso, ahora sobre los valores de las hojas en lugar de los coeficientes
- Ojo: el `lambda` del paquete es nuestro φ, no el shrinkage λ
- El paquete calcula la ganancia sin el $\frac{1}{2}$ del paper, así que `gamma` $= 2\zeta$

XGBoost: aproximación de segundo orden

- Expandimos la pérdida del paso m alrededor de $f_{m-1}(x_i)$ (Taylor de 2º orden):

$$L\big(y_i, f_{m-1}(x_i) + T_m(x_i)\big) \approx L\big(y_i, f_{m-1}(x_i)\big) + g_i\, T_m(x_i) + \tfrac{1}{2} h_i\, T_m(x_i)^2 \tag{28}$$

$$g_i = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}, \qquad h_i = \left[\frac{\partial^2 L(y_i, f(x_i))}{\partial f(x_i)^2}\right]_{f=f_{m-1}}$$

- g_i es el gradiente de gradient boosting; h_i es nuevo: la **curvatura** de la pérdida

XGBoost: pesos de las hojas y criterio de corte

- Esto hace que el valor óptimo de cada hoja (para un arbol dado)

$$w_j^* = -\frac{G_j}{H_j + \varphi} \tag{29}$$

- Al dividir una hoja en izquierda (L) y derecha (R), la mejora del objetivo es

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \varphi} + \frac{G_R^2}{H_R + \varphi} - \frac{(G_L + G_R)^2}{H_L + H_R + \varphi}\right] - \zeta \tag{30}$$

- ζ **poda**: solo se divide si Gain > 0, es decir, si la mejora supera el costo de la hoja nueva
- φ **encoge**

XGBoost: ¿Qué mejora?

- Controla el sobreajuste
  - ζ penaliza hojas: poda los cortes que no pagan su costo
  - φ y α encogen los valores de las hojas (Ridge y Lasso)
- Usa el gradiente y la curvatura
  - g_i da la dirección; h_i (la curvatura de la pérdida) da el tamaño del paso
  - Es un paso de Newton, con forma cerrada $w_j^* = -G_j/(H_j + \varphi)$
  - Sirve cualquier pérdida dos veces diferenciable: basta pasarle g y h
- Es rápido y escalable

Boosting + regularización + eficiencia computacional

Agenda

1. Recap
   - Problem at hand
   - CARTs
2. Bagging and Forests
   - Bagging
   - Random Forests
   - Out-of-Bag Error and Variable Importance
3. Boosting
   - AdaBoost.M1
   - Boosting Regression Trees
   - Gradient Boosting Trees
   - XGBoost
   - LightGBM

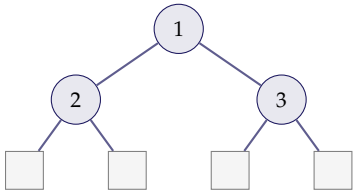
LightGBM: ¿Qué es?

- LightGBM (Ke et al. 2017) = gradient boosting optimizado para velocidad y memoria con n y p grandes
- El modelo es el mismo: $\hat{f}(x) = f_0 + \sum_{m=1}^{M} \lambda\, T_m(x)$, con g_i, h_i y las penalidades ζ, φ, α
- Cambia cómo se construye cada árbol. Son cuatro piezas:
  1. Histogramas: discretiza cada variable en bins y busca cortes entre bins, no entre observaciones
  2. Leaf-wise: divide la hoja con mayor Gain, sin importar la profundidad
  3. GOSS: usa más observaciones con gradientes grandes y menos con gradientes pequeños. Reduce el número de observaciones.
  4. EFB: combina variables que no toman valores distintos de cero al mismo tiempo (como dummies). Reduce el número de variables.

Source: Ke et al. (2017), *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, NeurIPS.

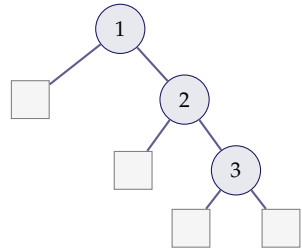
LightGBM: crecimiento leaf-wise

Level-wise (XGBoost default)



3 cortes, 4 hojas, profundidad 2.
El corte 3 se hace aunque
su Gain sea chico.

Leaf-wise (LightGBM default)



3 cortes, 4 hojas, profundidad 3.
Cada corte va a la hoja de mayor Gain.

Resumen: XGBoost vs LightGBM

- Ambos utilizan Gradient Boosting con árboles.
- Diferencias principales:
  - XGBoost crece por niveles; LightGBM, por hojas (por defecto).
  - LightGBM suele entrenar más rápido con datos grandes, pero puede ser más propenso al sobreajuste.
  - Ninguno domina sistemáticamente.

Example



photo from https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/